

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	22.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I (M.S.Dinesh / 192424106) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ M.S.Dinesh _____

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 Longest Palindromic Subsequence vs Word Break (String DP)

Given string "BBABCBCAB", find LPS length. Also check Word Break for "leetcode" with dictionary {"leet", "code"}. Compare DP tables and decisions. Analyze recurrence differences. Evaluate complexity. Generate insights on string DP problems.

Solution

1. Introduction

Longest Palindromic Subsequence (LPS)

- **Definition:** Finds the longest sequence of characters in a string that reads the same backward as forward, where characters do not need to be consecutive.
- **Core Goal:** Maximize the length of a matching subsequence by comparing characters from both ends of the string.
- **DP Category:** Two-dimensional intervals/substring DP.

Word Break Problem (WBP)

- **Definition:** Determines if a given string can be segmented into a space-separated sequence of one or more dictionary words.
- **Core Goal:** Verify feasibility (True/False) by finding valid split points within the string.
- **DP Category:** One-dimensional prefix/linear DP.

2. Problem Solution

Part A: LPS for "BBABCBCAB"

The length of the longest palindromic subsequence for "**BBABCBCAB**" is **7**.

One such maximum subsequence is "**BABCBAB**" (or "BACBCAB").

DP Table Construction

Let $dp[i][j]$ be the length of the LPS in the substring $(S[i \dots j])$.

$(i \setminus j)$ 0 (B) 1 (B) 2 (A) 3 (B) 4 (C) 5 (B) 6 (C) 7 (A) 8 (B)

0 (B)	1	2	2	3	3	5	5	5	7
1 (B)	0	1	1	3	3	5	5	5	5
2 (A)	0	0	1	1	1	3	3	3	3

3 (B)	0	0	0	1	1	3	3	3	3
4 (C)	0	0	0	0	1	1	3	3	3
5 (B)	0	0	0	0	0	1	1	3	3
6 (C)	0	0	0	0	0	0	1	1	1
7 (A)	0	0	0	0	0	0	0	1	1
8 (B)	0	0	0	0	0	0	0	0	1

Note: $dp[0][8] = 7$ yields our final answer.

Part B: Word Break for "leetcode"

- **Inputs:** String $S = \text{"leetcode"}$, Dictionary $D = \{\text{"leet"}, \text{"code"}\}$.
- **Result:** **True**, because the string can be cleanly segmented into "leet" + "code".

DP Array State Decisions

Let $dp[i]$ be a boolean representing if the prefix $(S[0 \dots i-1])$ can be segmented.

Base case: $dp[0] = \text{True}$.

- $dp[1]$ to $dp[3]$ ("l", "le", "lee"): **False** (no dictionary matches).
- $dp[4]$ ("leet"): **True**, because $dp[0]$ is True and "leet" is in D .
- $dp[5]$ to $dp[7]$ ("leetc", "leetco", "leetcod"): **False** (no valid tail matches).
- $dp[8]$ ("leetcode"): **True**, because $dp[4]$ is True and the remaining tail "code" is in D .

3. Algorithm

LPS Algorithm

1. **Initialize** a 2D array dp of size $N \times N$ with zeros.
2. **Base Case**: Set all diagonal elements $dp[i][i] = 1$ (every single character is a palindrome of length 1).
3. **Loop Structure**: Iterate through substring lengths from 2 to N , and sliding window starting positions i .
4. **Character Match**: If $S[i] == S[j]$, then set $dp[i][j] = dp[i + 1][j - 1] + 2$.
5. **Character Mismatch**: If $(S[i] \neq S[j])$, then set $dp[i][j] = \max(dp[i + 1][j], dp[i][j - 1])$.

Word Break Algorithm

1. **Initialize** a 1D boolean array dp of size $N + 1$ with `False`.
2. **Base Case**: Set $dp[0] = \text{True}$ to signify an empty string prefix is valid.
3. **Outer Loop**: Traverse the string forward from index $i = 1$ to N to build valid prefixes.
4. **Inner Loop**: Check all possible split positions j from 0 up to $i - 1$.
5. **Transition**: If $dp[j]$ is `True` and the substring $(S[j \dots i-1])$ exists in the dictionary, mark $dp[i] = \text{True}$ and break the inner loop.

4. About the Formula (Recurrence Relations)

LPS Formula

$$dp[i][j] = \begin{cases} dp[i+1][j-1] + 2 & \text{if } S[i] == S[j] \\ \max(dp[i+1][j], dp[i][j-1]) & \text{if } S[i] \neq S[j] \end{cases}$$

- **Two-Way Expansion**: The formula evaluates state changes shrinking inward from both the left boundary (i) and right boundary (j).
- **Dependency Structure**: To compute $dp[i][j]$, you must already know the values beneath it ($dp[i + 1]$) and to its left ($dp[j - 1]$).
- **Optimal Substructure**: The optimal solution for a larger substring explicitly depends on the optimal solution of its smaller internal sub-problems.
- **Max Value Accumulation**: Mismatches pass the highest value upward, ensuring non-continuous sequences are preserved.
- **Boundary Strictness**: It assumes $i \leq j$, leaving the lower half of the 2D grid unused.

Word Break Formula

$$\backslash(dp[i]=\bigvee_{j=0}^{i-1}\backslashleft(dp[j]\backslash\text{and}(S[j\dots i]\backslash\text{in}\backslash\text{Dictionary})\backslash\right)\backslash)$$

- **Linear Prefix Checking:** The formula relies on checking previous valid cutting spots (j) rather than shrinking boundaries.
 - **Logical OR Accumulation:** A state becomes `True` if *at least one* valid splitting choice satisfies the structural conditions.
 - **1D Dependencies:** Each index i depends exclusively on historical values stored in the same 1D array.
 - **Dictionary Constraints:** Subproblem progression is bounded strictly by the length constraints of words present in the dictionary.
 - **Directional Flow:** The algorithm evaluates strictly from left to right, solving prefixes sequentially.
-

5. Advantages

LPS Advantages

- **Character Skipping:** Handles highly noisy string patterns efficiently by skipping non-matching middle characters.
- **Substructure Reusability:** Prevents the exponential complexity of checking all 2^N possible subsequences via table lookups.
- **Exact Tracking:** The 2D table allows back-tracking to reconstruct the actual sequence string, not just its length.
- **Symmetry Optimization:** Can be modified easily to solve related problems like Longest Common Subsequence (LCS) by reversing the string.
- **Deterministic Steps:** Avoids arbitrary branching choices, resolving transitions in strict $O(1)$ calculations per cell.

Word Break Advantages

- **Memory Efficiency:** Utilizes a 1D storage array ($O(N)$ space complexity), which uses far less memory than LPS's 2D grid.
- **Early Termination:** The inner loop can break early the moment a single true split point is verified, saving computation time.
- **Dictionary Tuning:** Performance can be accelerated significantly by checking substrings up to the maximum word length in the dictionary.

- **Flexible Lookup Integration:** Works alongside fast hash sets or Trie structures to reduce dictionary search times to $O(1)$ or $O(W)$.
- **Scalable Pruning:** Easily adapts to print all valid sentence variations without rewriting the foundational DP table logic.

6. Limitations

LPS Limitations

- **High Space Overhead:** Requires an $O(N^2)$ table matrix, which becomes a bottleneck for exceptionally long string lengths.
- **Strict Quadratic Time:** Must compute every cell in the upper triangle; it cannot exit early even if a palindrome is found early.
- **Locality Issues:** Iterating through rows and columns in nested intervals causes poorer CPU cache locality than sequential 1D scans.
- **Initialization Dependency:** Fails to work efficiently without setting up precise base conditions across the entire matrix main diagonal.
- **No Word Meaning:** Completely blind to real-world word context; it processes structural patterns solely at the character level.

QUESTIONS

Q1

Longest Palindromic
Subsequence vs Word Break
(String DP)
100 marks

1/1 answered

Q1 Longest Palindromic Subsequence vs Word Break (String DP)

100 marks

Given string "BBABCB CAB", find LPS length. Also check Word Break for "leetcode" with dictionary ("leet", "code"). Compare DP tables and decisions. Analyze recurrence differences. Evaluate complexity. Generate insights on string DP problems.

Your Code

```
1 def lps_length(s: str) -> int:
2     n = len(s)
3     dp = [[0] * n for _ in range(n)]
4     for i in range(n):
5         dp[i][i] = 1
6     for length in range(2, n + 1):
7         for i in range(n - length + 1):
8             j = i + length - 1
9             if s[i] == s[j]:
10                dp[i][j] = 2 if length == 2 else dp[i + 1][j - 1] + 2
11            else:
12                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
13     return dp[0][n - 1]
14
15
16 def word_break(s: str, wordDict: set) -> bool:
17     n = len(s)
18     dp = [False] * (n + 1)
19     dp[0] = True
20     for i in range(1, n + 1):
21         for w in wordDict:
22             if i >= len(w) and dp[i - len(w)] and s[i - len(w):i] == w:
23                 dp[i] = True
24                 break
25     return dp[n]
26
27
28 s = "BBABCB CAB"
```

Input

stdin input

Output

7
True